

tf.compat.v1.mixed_precision.MixedPrecisionLossScaleOptimizer

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/mixed_precision/MixedPrecisionLossScaleOptimizer

An optimizer that applies loss scaling.

Function Signatures

```
tf.compat.v1.mixed_precision.MixedPrecisionLossScaleOptimizer(  
    opt, loss_scale )  
  
loss = ... loss *= loss_scale  
grads = gradients(loss, vars)  
grads /= loss_scale
```

Code Examples

```
tf.compat.v1.mixed_precision.MixedPrecisionLossScaleOptimizer(  
    opt, loss_scale  
)
```

```
tf.compat.v1.mixed_precision.MixedPrecisionLossScaleOptimizer(  
    opt, loss_scale  
)
```

```
loss = ...
loss *= loss_scale
grads = gradients(loss, vars)
grads /= loss_scale
```

```
loss = ...
loss *= loss_scale
grads = gradients(loss, vars)
grads /= loss_scale
```

```
apply_gradients(
grads_and_vars, global_step=None, name=None
)
```

```
apply_gradients(
grads_and_vars, global_step=None, name=None
)
```

```
compute_gradients(
loss,
var_list=None,
gate_gradients=optimizer.Optimizer.GATE_OP,
aggregation_method=None,
colocate_gradients_with_ops=False,
grad_loss=None
)
```

```
compute_gradients(
loss,
var_list=None,
gate_gradients=optimizer.Optimizer.GATE_OP,
aggregation_method=None,
colocate_gradients_with_ops=False,
grad_loss=None
)
```

Documentation

An optimizer that applies loss scaling.

Inherits From: [Optimizer](#)

View aliases

Compat aliases for migration

See

[Migration guide for more details.](#)

`tf.compat.v1.train.experimental.MixedPrecisionLossScaleOptimizer`

Loss scaling is a process that multiplies the loss by a multiplier called the loss scale, and divides each gradient by the same multiplier. The pseudocode for this process is:

Mathematically, loss scaling has no effect, but can help avoid numerical underflow in intermediate gradients when float16 tensors are used for mixed precision training. By multiplying the loss, each intermediate gradient will have the same multiplier applied.

The loss scale can either be a fixed constant, chosen by the user, or be dynamically determined. Dynamically determining the loss scale is convenient as a loss scale does not have to be explicitly chosen. However it reduces performance.

This optimizer wraps another optimizer and applies loss scaling to it via a `LossScale`. Loss scaling is applied whenever gradients are computed, such as through `minimize()`.

Raises

Methods

apply_gradients

[View source](#)

Apply gradients to variables.

This is the second part of `minimize()`. It returns an `Operation` that conditionally applies gradients if all gradient values are finite. Otherwise no update is performed (nor is `global_step` incremented).

compute_gradients

[View source](#)

Compute gradients of loss for the variables in `var_list`.

This adjusts the dynamic range of the gradient evaluation by scaling up the loss value. The gradient values are then scaled back down by the reciprocal of the loss scale. This is useful in reduced precision training where small gradient values would otherwise underflow the representable range.

get_name

[View source](#)

get_slot

[View source](#)

Return a slot named name created for var by the Optimizer.

Some Optimizer subclasses use additional variables. For example Momentum and Adagrad use variables to accumulate updates. This method gives access to these Variable objects if for some reason you need them.

Use `get_slot_names()` to get the list of slot names created by the Optimizer.

get_slot_names

[View source](#)

Return a list of the names of slots created by the Optimizer.

See `get_slot()`.

minimize

[View source](#)

Add operations to minimize loss by updating var_list.

This method simply combines calls `compute_gradients()` and

`apply_gradients()`. If you want to process the gradient before applying them call `compute_gradients()` and `apply_gradients()` explicitly instead of using this function.

eager compatibility

When eager execution is enabled, loss should be a Python function that takes no arguments and computes the value to be minimized. Minimization (and gradient computation) is done with respect to the elements of `var_list` if not `None`, else with respect to any trainable variables created during the execution of the loss function. `gate_gradients`, `aggregation_method`, `colocate_gradients_with_ops` and `grad_loss` are ignored when eager execution is enabled.

variables

[View source](#)

Returns the variables of the Optimizer.

Class Variables